

JDBC puro: conexión manual sin ORM

4. JDBC puro: conexión manual sin ORM

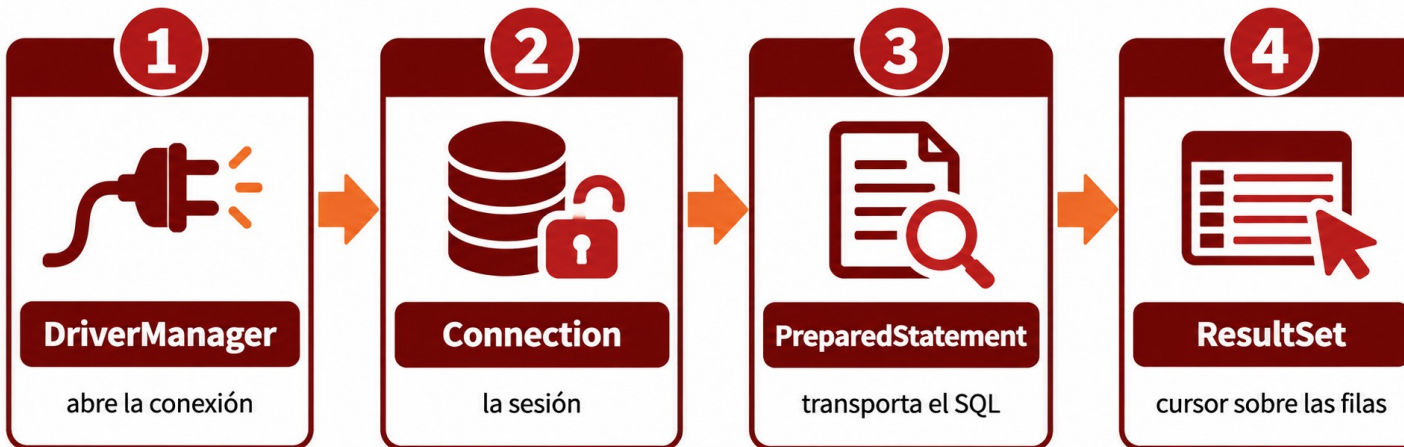
De Spring Data JPA a JDBC: las cuatro piezas

Hasta ahora: Spring Data JPA

Declaras una entidad, extiendes JpaRepository y save() o findAll() funcionan sin una línea de SQL ni gestionar conexiones.

Hoy: JDBC

La API estándar de Java para bases de datos relacionales y la base sobre la que se apoya todo lo demás, incluido Hibernate.



Cada pieza es un recurso

Mientras está abierta ocupa una conexión de red y memoria en el gestor. Si no se cierra, sigue ocupándolas hasta que algo la cierre, o hasta que termine el proceso.

La problemática: todo pasa a ser cosa tuya

- 1 Abrir y cerrar la conexión** Con la URL, usuario y contraseña correctos, y sin olvidarte de cerrarla.
- 2 Escribir el SQL a mano** Como texto: nadie comprueba la sintaxis hasta que lo ejecutas.
- 3 Pasar los parámetros con cuidado** O te expones a una inyección SQL.
- 4 Recorrer el resultado fila a fila** Extrayendo cada columna con el getXxx que toque y construyendo tú el objeto Java.
- 5 Gestionar SQLException** Una excepción checked: hay que capturarla o declararla en cada método que toque JDBC.

Ninguna es difícil por separado: el problema es acordarse de las cinco, en cada consulta, cada vez.

La misma consulta, dos formas

Con Spring Data JPA (lo que ya conoces)

```
public interface LibroRepository
    extends JpaRepository<Libro, Long>
{
    List<Libro> findByEditorialId(
        Long editorialId);
}
```

Una línea, sin cuerpo

Spring Data JPA lee el nombre (findBy + EditorialId) y genera el SQL él solo.

Quince líneas para lo mismo, y esta ya es la versión correcta, con los recursos cerrándose solos.

Con JDBC puro (lo que vas a hacer hoy)

```
String sql = "SELECT id, titulo, precio "
    + "FROM libro WHERE editorial_id = ?";

try (Connection conn = DriverManager
    .getConnection(url, user, pass);
    PreparedStatement stmt =
        conn.prepareStatement(sql)) {
    stmt.setLong(1, editorialId);
    try (ResultSet rs = stmt.executeQuery()) {
        List<Libro> libros = new ArrayList<>();
        while (rs.next()) {
            Libro libro = new Libro();
            libro.setId(rs.getLong("id"));
            libro.setTitulo(rs.getString("titulo"));
            libro.setPrecio(
                rs.getBigDecimal("precio"));
            libros.add(libro);
        }
        return libros;
    }
} catch (SQLException e) {
    throw new RuntimeException(
        "Error accediendo a la base de datos", e);
}
```

Abrir la conexión y preparar la sentencia

1. Abrir la conexión



2. Preparar la sentencia



3. Ejecutar y recorrer



4. Cerrar los recursos

Paso 1: abrir la conexión

```
Connection conn =  
    DriverManager.getConnection(  
        "jdbc:postgresql://localhost:5432/libreria_db",  
        "libreria_user",  
        "password123"  
    );
```

Paso 2: preparar la sentencia

```
String sql = "SELECT id, titulo, precio "  
    + "FROM libro WHERE editorial_id = ?";  
PreparedStatement stmt =  
    conn.prepareStatement(sql);  
stmt.setLong(1, editorialId);
```

Cada llamada abre una conexión nueva

La misma URL, usuario y contraseña de application-dev.yml, pero pasados desde Java. Spring usaría un pool (HikariCP); aquí, DriverManager para ver JDBC sin nadie que gestione las conexiones.

Nunca concatenes el SQL

Con "... = " + editorialId, un valor como 1 OR 1=1 se interpreta como código: inyección SQL. Con ? el valor viaja aparte y el driver lo trata siempre como dato.

Recorrer el resultado y cerrar los recursos

1. Abrir la conexión



2. Preparar la sentencia



3. Ejecutar y recorrer



4. Cerrar los recursos

Paso 3: ejecutar y recorrer

```
ResultSet rs = stmt.executeQuery();
while (rs.next()) {
    Long id = rs.getLong("id");
    String titulo = rs.getString("titulo");
    BigDecimal precio =
        rs.getBigDecimal("precio");
    // mapear a mano a un objeto Java
}
```

ResultSet es un cursor

rs.next() avanza fila a fila; getXxx("columna") extrae cada valor.

Este mapeo, de fila a objeto Libro, es exactamente lo que Hibernate hace por ti, en cada fila.

Paso 4: try-with-resources

```
try (Connection conn = DriverManager
    .getConnection(url, user, pass);
    PreparedStatement stmt =
        conn.prepareStatement(sql)) {
    stmt.setLong(1, editorialId);
    try (ResultSet rs = stmt.executeQuery()) {
        while (rs.next()) { /* ... */ }
    }
}
```

Cerrar no es opcional

Cierra cada recurso al salir del bloque, en orden inverso y también si hay excepción. Si no, la sesión con PostgreSQL sigue ocupada y, repetido, se alcanza el límite de conexiones.

¿Y para escribir? INSERT con JDBC puro

Con Spring Data JPA

```
Libro guardado =  
    libroRepository.save(libro);  
Long idGenerado = guardado.getId();  
// ya viene relleno
```

save() lo hace todo

Construye el INSERT, lo ejecuta y rellena el id de tu objeto con el valor generado.

executeUpdate(), no executeQuery()

INSERT, UPDATE y DELETE devuelven el número de filas afectadas, no un ResultSet.

Con JDBC puro

```
String sql = "INSERT INTO libro (titulo, precio,"  
    + " editorial_id) VALUES (?, ?, ?)";  
  
try (Connection conn = DriverManager  
    .getConnection(url, user, pass);  
    PreparedStatement stmt = conn.prepareStatement(  
        sql, Statement.RETURN_GENERATED_KEYS)) {  
    stmt.setString(1, titulo);  
    stmt.setBigDecimal(2, precio);  
    stmt.setLong(3, editorialId);  
    stmt.executeUpdate();  
    try (ResultSet keys = stmt.getGeneratedKeys()) {  
        if (keys.next()) {  
            Long idGenerado = keys.getLong(1);  
        }  
    }  
}
```

Fácil de olvidar: RETURN_GENERATED_KEYS

Sin él, getGeneratedKeys() devuelve un ResultSet vacío y no sabes qué id tiene la fila nueva.

Transacciones manuales con JDBC

Una promoción aplica un descuento a dos libros. No tendría sentido que el primero cambiara de precio y el segundo no, porque un error a mitad del proceso haya cortado el resto.

Por defecto: autocommit

Cada sentencia se confirma automáticamente en cuanto termina.

Con `setAutoCommit(false)`

Tú decides cuándo confirmar o deshacer los cambios: `commit()` o `rollback()`.

Para agrupar varias operaciones en una transacción, primero desactiva el autocommit:

```
conn.setAutoCommit(false);
```

O se completan todas las operaciones, o no se conserva ninguna.

Transacción completa: dos libros, commit y rollback

Primer libro

```
String sql =
    "UPDATE libro SET precio = precio * ?"
    + " WHERE id = ?";

try (Connection conn = DriverManager
    .getConnection(url, user, pass)) {
    conn.setAutoCommit(false);
    try (PreparedStatement stmt =
        conn.prepareStatement(sql)) {
        // Primer libro
        stmt.setBigDecimal(1,
            new BigDecimal("0.90"));
        stmt.setLong(2, 1L);
        if (stmt.executeUpdate() != 1) {
            throw new SQLException(
                "No se ha encontrado el primero");
        }
    }
}
```

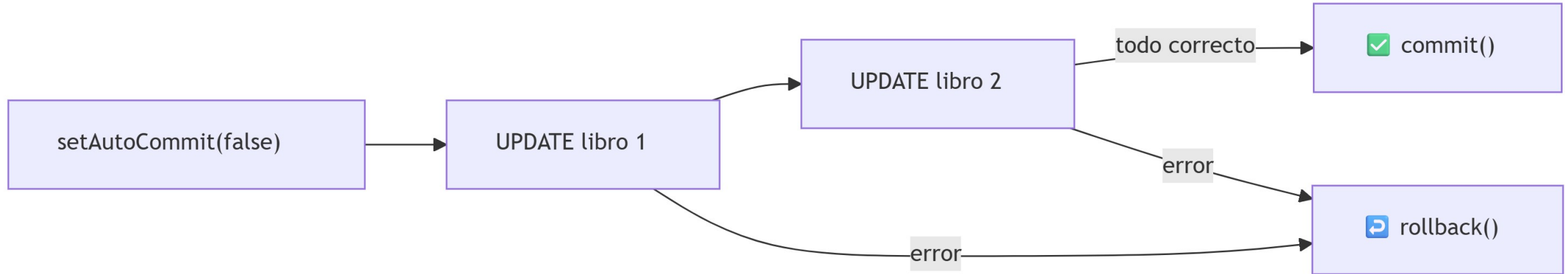
Segundo libro, commit y rollback

```
// Segundo libro
stmt.setBigDecimal(1,
    new BigDecimal("0.90"));
stmt.setLong(2, 2L);
if (stmt.executeUpdate() != 1) {
    throw new SQLException(
        "No se ha encontrado el segundo");
}

conn.commit();
} catch (SQLException e) {
    conn.rollback();
    throw e;
} catch (SQLException e) {
    throw new RuntimeException(
        "No se ha podido aplicar la promoción", e);
}
```

Un UPDATE sin fila no siempre lanza excepción: `executeUpdate()` devuelve 0. Por eso el código comprueba que cada sentencia modifique exactamente una fila.

El flujo, y la diferencia con @Transactional



Con JDBC puro

Abrir la transacción, hacer commit y ejecutar el rollback forman parte de tu propio código.

Con @Transactional

Spring abre la transacción, hace el commit y ejecuta el rollback por ti.

Si algo falla en la segunda actualización, rollback() deshace también la primera: dentro de la misma transacción, o todo o nada.

Lo que Spring Data JPA te estaba ahorrando

Con JDBC puro, tienes que...	Con Spring Data JPA...
Pedir una Connection a DriverManager y cerrarla al terminar	Spring toma una conexión del pool de HikariCP y la devuelve al terminar
Escribir el SQL como texto, tú mismo	Hibernate lo genera a partir de tus entidades y del nombre del método
Mapear cada fila del ResultSet a un objeto	Hibernate mapea cada fila a tu entidad automáticamente
Capturar SQLException en cada método	Spring la convierte en excepciones unchecked, más cómodas de propagar
Desactivar el autocommit y decidir commit() o rollback()	@Transactional abre, confirma o deshace la transacción por ti

¿Por qué aprender JDBC si el ORM lo hace todo?

Ahora que has visto la «fontanería» real, `libroRepository.findAll()` deja de ser magia: es JDBC con todo este trabajo repetitivo automatizado.

Un ORM sigue generando SQL

Entender qué SQL genera, y por qué a veces conviene evitarlo, te ayudará a diagnosticar problemas de rendimiento. Y hay casos (consultas muy específicas, procedimientos almacenados) donde JDBC o `JdbcTemplate` son lo más directo.

Lo que viene: Actividad 1.3

Dejas GameVault un momento: un proyecto Java suelto, sin Spring, con su propia base de datos, para ver JDBC sin nada alrededor que lo oculte.

Ideas clave

1

JDBC es la API estándar de Java para bases de datos y la base sobre la que se apoya cualquier ORM, incluido Hibernate.

2

Cuatro piezas: DriverManager/DataSource (abre), Connection (sesión), PreparedStatement (SQL) y ResultSet (cursor).

3

Con JDBC puro te encargas de abrir y cerrar la conexión, escribir el SQL, mapear cada fila y capturar SQLException.

4

Un recurso abierto y no cerrado sigue consumiendo: se cierra siempre, con try-with-resources.

5

PreparedStatement con parámetros (?) evita la inyección SQL; concatenar datos externos con + es peligroso siempre.

6

Escribir: executeUpdate() y Statement.RETURN_GENERATED_KEYS si necesitas el id generado.

7

Transacción manual: setAutoCommit(false), commit() si todo va bien y rollback() si algo falla.

8

@Transactional y JpaRepository hacen esto mismo por ti: abrir, commit y rollback.